



Learn Web Development with AI Part 1:Post Read

Links to resources used in the session:

- [Session Prompts and Resources \(Google Drive\)](#)
-

1.0 Introduction to Web Development with AI

The integration of Artificial Intelligence (AI) into web development represents a significant paradigm shift. Traditionally, building a functional, aesthetically pleasing, and responsive website required extensive knowledge of coding languages, debugging, and framework architecture. With the advent of AI-driven Integrated Development Environments (IDEs) and Large Language Models (LLMs), the barrier to entry has been drastically lowered. Developers and non-technical individuals alike can now generate production-ready code by providing structured natural language prompts.

1.1 The Core Triad of Web Technologies

Before leveraging AI to generate code, it is crucial to understand the foundational technologies that construct a web page. Every modern website relies on three primary languages, each serving a distinct purpose in the architecture of the site.

Technology	Primary Function	Human Body Analogy	Practical Example
HTML (HyperText Markup Language)	Defines the structure and content of the web page.	The Skeleton (provides the foundational framework and structure).	Creating a paragraph of text or inserting an image placeholder.
CSS (Cascading	Dictates the visual presentation,	The Skin and Muscles	Changing text color to red, adding a

Technology	Primary Function	Human Body Analogy	Practical Example
Style Sheets)	layout, and design.	(determines appearance, color, and physical layout).	background gradient, or rounding the corners of a box.
JavaScript (JS)	Enables interactivity, dynamic content, and complex logic.	The Brain and Nervous System (controls behavior and responses to stimuli).	Triggering a pop-up alert when a button is clicked or validating a contact form.

2.0 Development Environment and Tools

To build and preview a website locally, an Integrated Development Environment (IDE) is required. While traditional text editors suffice for manual coding, AI-assisted development relies on specialized tools.

2.1 Cursor vs. Visual Studio Code (VS Code)

Cursor is an AI-first code editor built on the same foundation as VS Code. It features a built-in AI agent capable of reading the entire project context, generating multi-file code, and debugging errors automatically. **VS Code** is a highly extensible, industry-standard IDE that can also utilize AI through plugins (like GitHub Copilot).

Strategic Workflow: AI tools often operate on a freemium model with monthly usage limits (context tokens). A practical workaround for managing these limits during intensive development is to initiate the project and generate heavy architectural code using Cursor. Once the free tier limit is reached, the local project folder can be seamlessly opened in VS Code to continue manual refinement or utilize alternative free AI extensions.

3.0 Principles of Effective Prompt Engineering for UI/UX

The quality of the AI-generated website is directly proportional to the clarity, specificity, and structure of the prompts provided. Treating the AI as a highly capable but literal-minded assistant is essential.

3.1 Rules for Structuring Prompts

1. **Avoid Subjective Adjectives:** Terms like "make it look nice" are ambiguous. Instead, define exact visual parameters (e.g., "add a white background, rounded corners, and a subtle drop shadow").
 2. **Provide Contextual Personas:** Clearly define the subject of the website. Instead of "I am a web designer," use "I am a UX designer with 5 years of experience specializing in digital experiences."
 3. **Consolidate Instructions:** Combine multiple related instructions into a single, comprehensive prompt to maintain context and reduce token usage.
 4. **Iterative Conversation:** Treat prompting as a dialogue. If an element is incorrect, provide specific feedback (e.g., "The font size on the mobile view is too small, increase it to 16px").
 5. **Demand Explanations:** To facilitate learning, instruct the AI to explain the generated code using plain English and to insert explanatory comments directly into the codebase.
-

4.0 Step-by-Step Website Construction using AI

The process of building a multi-page portfolio website involves sequential prompting, starting from the foundational architecture to specific page components.

4.1 Project Initialization and Architecture

The first step is establishing the file structure. A structured prompt should instruct the AI to create a project folder containing the three essential files: `index.html`, `styles.css`, and `script.js`. Furthermore, the prompt must explicitly state that these files need to be linked together (e.g., the HTML file must reference the CSS stylesheet and the JavaScript file).

4.2 Building the Navigation System

A navigation bar is critical for user routing. The prompt should specify:

- **Links required:** Home, About, Projects, Contact.
- **Styling:** Dark background, white text, evenly spaced elements.
- **Behavior:** "Sticky" positioning (the bar remains at the top of the viewport during scrolling) and hover effects.
- **State indication:** The active page link should be highlighted to orient the user.

4.3 Designing the Hero Section

The "Hero" section is the prominent, above-the-fold area of a landing page. It serves as the primary hook for the user.

- **Typography:** A large headline introducing the persona, followed by a descriptive sub-headline.
- **Call to Action (CTA):** A distinct button (e.g., "See My Work") that links directly to the Projects section.
- **Aesthetics:** A complex background, such as a blue and purple gradient, to create visual interest.

4.4 Implementing the Skills Section (Grid Layouts)

To display competencies, a card-based grid layout is highly effective. The prompt should dictate:

- **Structure:** A responsive grid of cards.
- **Content per card:** A relevant emoji/icon, the skill name, and a one-sentence description.
- **Interactivity:** A hover effect that slightly lifts the card, providing tactile feedback to the user.

4.5 Developing the About Page

Creating a new page (`about.html`) requires instructing the AI to reuse the existing navigation bar and stylesheet to maintain consistency. The layout can be specified as a two-column design: a placeholder profile image on the left, and biographical text on the right. Additionally, a chronological timeline component can be requested to display career milestones.

4.6 Constructing the Projects Gallery

The `projects.html` page requires a gallery layout. The prompt should specify a grid of project cards, where each card contains a placeholder image, a title, a brief description, and small tags indicating the tools used (e.g., Figma, HTML). A "View Case Study" button should be included in each card.

4.7 Creating Functional Contact Forms

The `contact.html` page introduces user input elements. The prompt must detail the exact fields required:

- Full Name (Text input)
- Email Address (Email input)

- Subject (Dropdown menu with specific options like "Project Inquiry" or "Collaboration")
- Message (Text area)

Crucially, the prompt should request focus states (visual changes when a user clicks into a field) and a success message that appears upon submission, which relies on JavaScript logic.

4.8 Standardizing the Footer

A footer anchors the bottom of every page. The prompt should instruct the AI to append a consistent footer across all four HTML files, containing social media links, a copyright notice, and redundant navigation links, styled with a dark background for contrast.

5.0 Refinement and Optimization

Once the structural components are in place, the website requires global refinement to ensure professional quality and usability across all devices.

5.1 Typography and Visual Hierarchy

Visual hierarchy guides the user's eye to the most important information first. A global refinement prompt can instruct the AI to:

- Import modern web fonts (e.g., via Google Fonts).
- Establish a strict typographic scale (e.g., 48px for Hero headlines, 32px for section headers, 24px for subheadings).
- Increase line spacing (leading) to improve readability.
- Alter global color schemes (e.g., shifting from a purple/blue theme to a green/slate theme) by updating CSS variables.

5.2 Responsive Web Design (RWD)

A website must function seamlessly on desktops, tablets, and mobile phones.

Responsiveness is achieved through CSS media queries. The AI should be prompted to review all pages and ensure mobile compatibility. Specific instructions should include collapsing the standard navigation bar into a "hamburger menu" on smaller screens and ensuring grid layouts stack vertically on mobile devices.

Testing Responsiveness: Developers can test mobile views locally using a web browser's built-in Developer Tools. By right-clicking the page, selecting "Inspect," and toggling the

"Device Toolbar," users can simulate how the site renders on various specific devices (e.g., iPhone 14 Pro, iPad Mini).

Summary and Key Takeaways

- **AI Democratizes Development:** AI IDEs like Cursor allow individuals without formal coding backgrounds to generate complex, multi-page websites using natural language.
 - **Prompt Precision is Paramount:** Vague requests yield poor results. Prompts must dictate exact layouts, behaviors, color schemes, and structural relationships.
 - **Understand the Underlying Triad:** Even when using AI, knowing that HTML provides structure, CSS provides style, and JS provides logic is essential for debugging and iterative prompting.
 - **Modularity and Consistency:** Building a site page-by-page requires explicit instructions to the AI to reuse existing stylesheets and navigation components to maintain a cohesive user experience.
 - **Responsiveness is Mandatory:** A modern website must be explicitly prompted to be responsive, ensuring usability across all device viewports.
-

Self-Check Questions

1. Explain the distinct roles of HTML, CSS, and JavaScript in web development using the human body analogy.
 2. Why is the prompt "make the website look professional" considered ineffective, and how should it be rewritten to yield better AI-generated code?
 3. What is the purpose of a "Hero" section on a website, and what key elements should it contain to maximize user engagement?
 4. Describe the concept of Responsive Web Design (RWD) and explain how a developer can test it locally without deploying the website to a live server.
 5. If an AI agent reaches its monthly token limit in an IDE like Cursor, what is a practical workaround to continue developing the local project?
 6. Why is it necessary to explicitly instruct the AI to link the `styles.css` and `script.js` files to the `index.html` file during project initialization?
-

Additional Learning Resources and Practice

- Practice writing structured prompts by taking a screenshot of a favorite website and describing its layout, colors, and typography in plain English.

- Explore browser Developer Tools (Inspect Element) to understand how CSS properties affect live web pages.
- Investigate the concept of CSS Media Queries to understand the underlying code that makes websites responsive to different screen sizes.
- Research version control systems like GitHub, which are essential for moving a local website project to a live, cloud-hosted environment.